31927 - Applications Development with .NET
32998 - .NET Applications Development
SPRING 2026
ASSIGNMENT-2 COVER SHEET

PROJECT TITLE	ASSIGNMENT-2: Group Project
LEAD STUDENT NAME	Benjamin Walters
LEAD STUDENT ID	24971095
GROUP MEMBER NAME(S) AND STUDENT ID(S)	Benjamin Walters 24971095 Seniru Basnayake 24796500 Aidan Ramen 24749855
SUBMISSION DATE	19/10/2026

AI USE RESTRICTIONS FOR PROGRAMMING WORK

Do not use AI to generate a substantial or complete solution to the programming assignment, or to bypass the learning outcomes of the assessment.

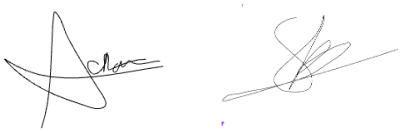
- AI must not be used to invent or misrepresent code execution, test results, screenshots, datasets, performance measurements, external libraries/APIs used, contributions of group members, or any other project detail.
- Permitted support may include concept explanations, syntax help, debugging suggestions, code review, test-case ideas, partial code generation, documentation/comments, and limited code examples where consistent with the assessment instructions.
- You must understand, review, run and test any AI-assisted code yourself, verify that it meets the specification, and be able to explain every submitted component and design decision.
- All AI use must be disclosed. AI output is not an authoritative source and does not replace reading subject materials, documentation, or instructions from teaching staff.

DECLARATION OF ORIGINALITY AND RESPONSIBILITY

☑ I confirm that the submitted work is the work of the named student(s), except where collaboration, source code, libraries, data, or AI assistance has been appropriately acknowledged. I understand the submitted program and can explain its implementation, testing and results.

Lead student signature: 

Date: 19/10/2026

Group member signature(s): 

Date: 19/10/2026

AI USE STATEMENT — ASSIGNMENT-2

Attach this completed statement as an appendix to your submission. Disclose all AI-enabled tools used, including uses that did not produce code retained in the final submission.

Student name(s) and ID(s)	Benjamin Walters (24796500) Seniru Basnayake (24796500), Aidan Ramen (24749855)
Subject code	☑ 31927 ☐ 32998
Project / assessment title	ASSIGNMENT-2: Group Project

☑ I did use any AI-enabled tool for this assessment. (If selected, proceed to the declaration.)

1. AI tools and model names

List every AI-enabled tool used (for example ChatGPT, GitHub Copilot, Microsoft Copilot, Gemini, Claude, Grammarly or other coding assistants). Record the model name shown in the tool, or "not shown".

AI tool or product	Model name (as shown)	Date(s) used
Gemini	3.6 Flash	29/09/2026
Gemini	3.6 Flash	04/10/2026
Gemini	3.6 Flash	19/10/2026

2. What the AI tools were used for (tick all that apply)

☐ Explained programming concepts or APIs	☐ Suggested algorithms, approaches or pseudocode
☐ Helped with syntax or small code examples	☐ Generated or completed code
☑ Debugged errors or explained error messages	☐ Reviewed/refactored code I had written
☐ Suggested unit tests, edge cases or test data	☑Generated documentation, comments or README text
☐ Helped with data analysis or visualisation code	☐ Assisted with version-control / build / tooling commands
☐ Proofread or revised report text	☑ Other: Helping Ideation of the actual project topic

3. Representative examples of AI use

Give 2–3 examples. Summarise what you asked the tool to do and explain what you accepted, changed, tested or rejected. Do not paste full chat logs unless requested.

Tool / model	Prompt or task summary	How I used, changed or rejected the output	Where it affected the submission
Gemini 3.6 Flash	Give us some loose ideas on a problem or issue that students may have when it come to university	Used the suggestions as inspiration and selected ideas that were relevant to university students and our app	Helped shape the initial ideas and features for the CampusHaps student social platform
Gemini 3.6 Flash	What type of comments should be put into a controller file to explain how it works?	Used the advice to understand where comments could explain the purpose of endpoints and important parts of the controller	Helped guide code commenting in the ASP.NET Core backend
Gemini 3.6 Flash	This method is giving me an error, ive tried to troubleshooting it with the built in fix but nothing seems to be working, what should I check for to resolve it?	Used the suggestions as a checklist to investigate possible causes and identify what needed further checking	Supported debugging and testing of the backend code during development

4. Verification and validation

☑ Ran and tested all submitted code and reviewed outputs, errors and edge cases	☑ Checked AI-suggested APIs/libraries against official documentation or course materials
☑ Confirmed that reported outputs, screenshots and performance results came from my/our actual program execution	☑ Reviewed AI-assisted code for correctness, security, originality, licensing/attribution and assessment requirements
☑ I/we can explain every submitted code component and justify the implementation and testing decisions	☑ No confidential, personal or unauthorised third-party material was uploaded to an AI tool

Briefly describe one specific check you performed and any change made as a result:

We tested the backend endpoints using Swagger to check that requests worked correctly and that the API returned the expected responses. I also checked how the code handled missing records and invalid input

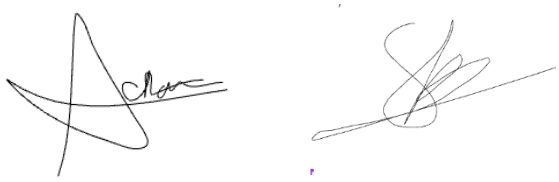
Declaration

I/we confirm that this statement accurately describes all AI-enabled tools used for this assessment and the extent to which their outputs influenced or appear in the submission. I/we followed the assessment instructions, independently reviewed and tested the submitted work, and take responsibility for the final code, report and results. I/we understand that prompts, outputs, commit history or other supporting evidence may be requested for an academic integrity review.

Lead student signature: Date: 19/10/2026



Group member signature(s): Date: 19/10/2026



31927 - Applications Development with .NET
32998 - .NET Applications Development

Project report

Seniru Basnayake 24796500 | Benjamin Walters 24971095 | Aidan Ramen 24749855

1. Introduction and Summary

1.1 Project Overview

CampusHaps is a platform for university/college students to interact with social events and gatherings. It's a central place for societies, clubs, and their events to congerate and keep updated. Students are able to use the platform to visit different clubs and view their posts and comments, whilst also being able to browse upcoming events in a feed or calendar, vote and comment on posts as well. The program is split into 2 major parts; Backend - which consists of C# ASP.NET Core Web API with a SQLite database and Frontend- which is built with SvelteKit.

1.2 Purpose and Problem Statement

University societies run a wide range of events, however, information around them is scattered. Students are limited to using external social media platforms and navigating through multiple different platforms and pages to communicate with other society members.

CampusHaps aims to address this problem by introducing a social platform where students can comment and vote on any upcoming university events, without having to go through the hassle of finding each club's social media page. This allows for a social space to discuss information relevant to the event or casual conversations to find like-minded students. Students can also browse upcoming events in a single feed or calendar view, and visit each club's community page to see its posts and events.

1.3 Key Features

1.3.1 Event Feed

A feed-style view displaying upcoming campus events.

- Browse and discover new events.
- Upvote events.
- Comment on events.
- Link users to event tickets.
- Switch between feed and calendar views.
- Filtering events by type
- Create and manage events
- Event status - ongoing or complemented

1.3.2 Event Calendar

A calendar view that allows students to explore events by date and see what's happening on campus.

1.3.3 Clubs and Societies

Each club or society has its own community page where members can connect and share content.

- Dedicated page for each club or society.
- A space for members to interact and engage with their community.
- Ability to promote and share upcoming events.

1.3.4 User Accounts

- Students are required to register with a university email address, only .edu.au emails are accepted
- Login with an email and passwords that are hashed.

2. Development Approach

2.1 Methodology

The application is built on a C# backend with a SvelteKit frontend that communicate between each other with a REST API. Controllers are used to handle the web requests, services are used to contain the business logic and the Entity Framework Core maps the models to database tables, translating LINQ queries into SQL. DTOs control what data is sent to and received by the frontend.

The workload is split across 3 members, Seniru Basnayake and Aidan Ramen have worked on the backend with a 50/50 split, and Benjamin Walters worked on the frontend. The split

2.2 Frameworks, Libraries and Tools

Frameworks:

- .NET9/C# - backend language and runtime
- ASP.NET Core Web API - Handles the HTTP request through controllers and returns JSON.
- Entity framework core 9 - Maps C# models to database tables to allow the database to be queried with LINQ
- SvelteKit (Svelte , TypeScript) - Frontend framework

Database:

- SQLite - File-based database to store users, events, clubs, comments, posts and votes.

Libraries:

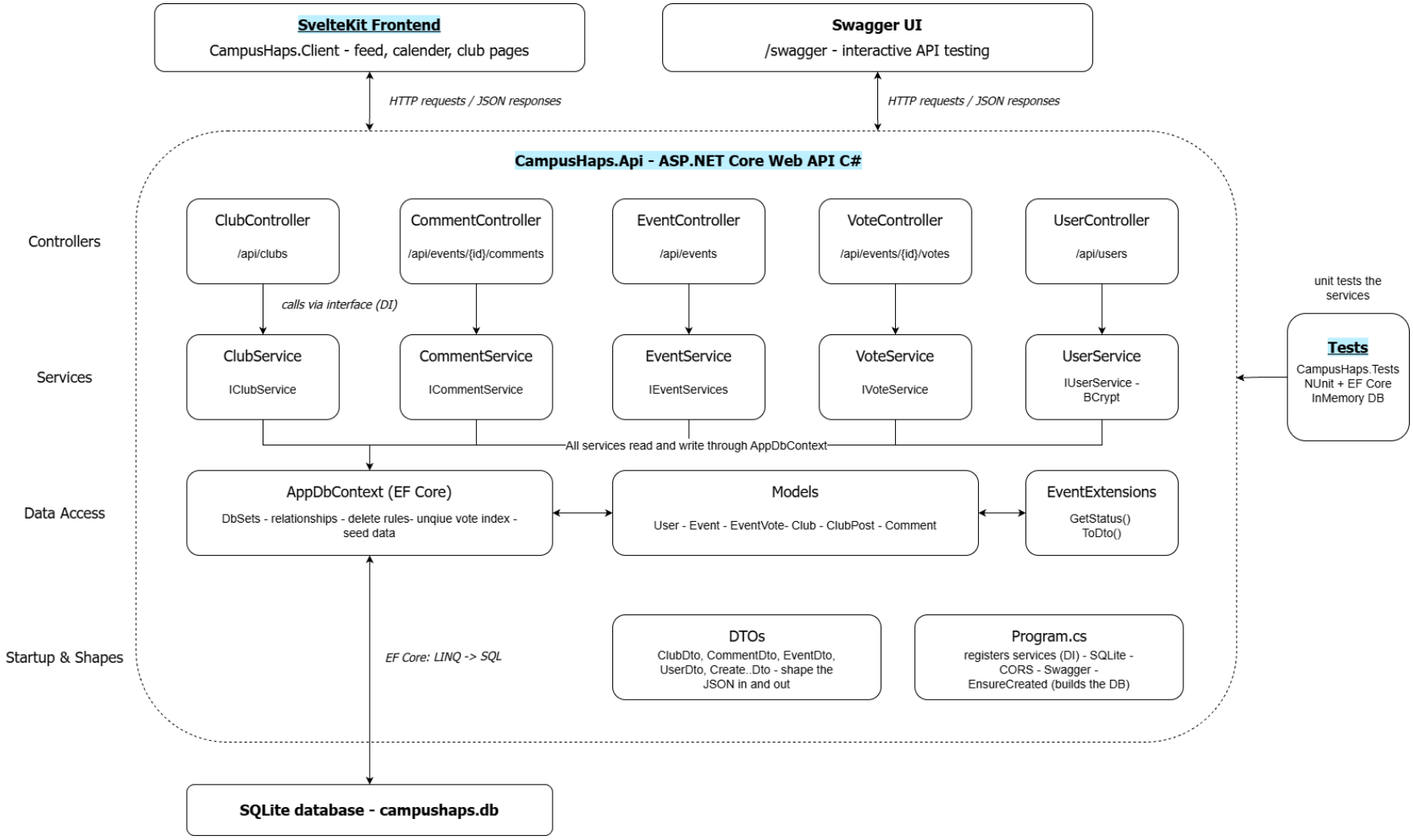
- BCrypt.Net-Next - Used to hash the passwords. Hashes are stored and not the plain text password
- Swagger - Used to test the backend through an interactive web page
- NUnit, NUnit3TestAdapter, Microsoft.NET.Test.Sdk - Used for unit testing
- Microsoft.EntityFrameworkCore.InMemory - Temporary database that is stored in memory, used by the unit tests.

Tools:

- Visual Studio 2022 - IDE used to develop the backend.
- Git and GitHub - Used to collaborate on the project.

3. Flowchart

3.1 System Architecture Diagram



The SvelteKit frontend sends HTTP requests to the ASP.NET Core Web API and receives JSON responses. Swagger is used to test the same endpoints during development. Inside the API, each controller handles the requests for one feature and calls its service through an interface, using dependency injection set up in Program.cs. The services contain the business logic and use AppDbContext to read and write data. AppDbContext uses Entity Framework Core to map the models to tables in the SQLite database and to translate LINQ queries into SQL. DTOs control the data sent to and from the frontend. The NUnit test project tests the services directly using an in-memory database.

4. Role of Team Members

4.1 Brief Descriptions of duties

Aidan Ramen - Responsible for the backend events, voting and users. Designed the Event,User and EventVote models as well as the EventType and EventStatus enums. Implemented the DTOs for sending and receiving event,vote and user data. Completed the interfaces, IEventService, IVoteService and IUserService and their services. This covers the event filtering, calendar date-range query, input validation and BCrypt password hashing as well as the EventExtensions class to calculate the event's status from its dates. Completed the Event, Vote and User controllers to allow communication between the frontend and backend. Wrote the NUnit tests for the services listed above.

Seniru Basnayake - Responsible for the backend clubs, club posts and comments, as well as the database setup. Designed the Club, ClubPost and Comment models and set up the AppDbContext, which defines the database tables, the relationships and delete rules between them, the rule that stops duplicate votes, and the seed data. Implemented the DTOs for sending and receiving club, post and comment data. Completed the interfaces, IClubService and ICommentService, and their services. This covers checking that clubs, events and users exist, rejecting empty posts and comments, sorting posts and comments by date, and making sure users can only edit or delete their own comments. Completed the Club and Comment controllers to allow communication between the frontend and backend, and set up Program.cs to connect the database, register the services and allow the frontend to call the API. Wrote the NUnit tests for the services listed above.

Benjamin Walters - Responsible for the development of the frontend user interface and API integration. Designed and implemented core client routing through Svelte 5 and its runes, including Event listing, Club feed, Calendar, Login page (with registration functionality). Ensured local state persistence through writable stores. Integrated API controllers for frontend verification/authorisation, handling endpoint authentication, form data validation, and account domain regulations.

4.2 Team Contributions Summary

Team Member	Files Completed
Aidan Ramen	**Enums** - EventType.cs , EventStatus.cs **Models** - Event.cs, User.cs, EventVote.cs **DTOs** - EventDto.cs, CreateEventDto.cs, VoteDto.cs, UserDto.cs, RegistorDto.cs, LoginDto.cs **Extensions** - EventExtensions.cs **Interfaces -** IEventService.cs, IVoteService.cs, IUserService.cs **Services -** EventService.cs, VoteService.cs, UserService.cs **Controllers -** EventController.cs, VoteController.cs , UserController.cs **Tests** - EventServiceTests.cs, VoteServiceTests.cs, UserServiceTests.cs
Seniru Basnayake	**Models** - Club.cs, ClubPost.cs, Comment.cs **Data** - AppDbContext.cs **DTOs** - ClubDto.cs, CreateClubDto.cs, ClubPostDto.cs, CreateClubPostDto.cs, CommentDto.cs, CreateCommentDto.cs **Interfaces** - IClubService.cs, ICommentService.cs **Services** - ClubService.cs, CommentService.cs **Controllers** - ClubController.cs, CommentController.cs **Startup** - Program.cs **Tests** - ClubServiceTests.cs, CommentServiceTests.cs
Benjamin Walters	**CampusHaps.Client** (**Lib** and **Routes**) **Api** - client.ts, clubs.ts, comments.ts, events.ts, index.ts **Components** - **Clubs** - ClubCard.svelte, ClubList.svelte, CreateClubModal.svelte **Components** - **Events** - EventCard.svelte, UpvoteButton.svelte **Components** - **Posts** - CreatePostModal.svelte, PostCard.svelte, PostFeed.svelte **Components** - Footer.svelte, LoadingSpinner.svelte, Modal.svelte, NavBar.svelte, CreateEventPost.svelte **Stores** - notifStore.ts, userStore.ts **Lib** - index.ts **Calendar** - +page.svelte **Clubs** - +page.svelte **Clubs** - **[id]** - +page.svelte **Events** - +page.svelte **Events** - **[id]** - +page.svelte **Login** - +page.svelte **Routes** - +layout.svelte, +page.svelte